KEYSTONE: Readiness Audits for Dispute-Key Availability in Encrypted Rollups

Journal Title
XX(X):1–23
©The Author(s) 0000
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/
SAGE

[Author details removed for peer review]

Abstract
Encrypted optimistic rollups can keep ciphertexts and execution receipts available and still fail a dispute because the threshold service that must decrypt the disputed record is unavailable, late or uncooperative. We call the missing property dispute-key availability (DKA) and separate it from ciphertext availability, valid-share registration, audit-time readiness, authorized decryptability and deadline liveness. KEYSTONE audits readiness with sampled canary shares, in the style of threshold-coin shares, carrying Chaum–Pedersen proofs; their transcripts are simulatable from public shares, so audits exercise real shares without revealing any record. We give exact operating characteristics for uniform and stratified samples, a placement condition and a deadline budget. We show that a single small audit cannot separate a committee one share short of its threshold from a live one, whereas a sequential test across audits can, and that confidentiality against parties who obtain other records' releases requires binding each release to its record. Exact analysis, simulation and a single-host 32-process testbed with ffdhe2048 cryptography confirm these separations, show that selective withholders pass every audit while defeating every dispute, and show that serial proof verification bounded completion time in our unoptimized prototype.

Keywords
Dispute-key availability, threshold decryption, encrypted rollups, readiness audit, acceptance sampling, selective withholding

1 Introduction

Optimistic rollups execute transactions off chain and accept the result unless a challenger proves it wrong within a dispute window[1]. When the executed workload is private, as in encrypted machine-learning inference[2,3], the records a challenger needs are published only as ciphertexts. Data-availability mechanisms can keep these ciphertexts retrievable. They cannot ensure that the committee holding shares of the decryption key delivers enough valid decryption contributions once a dispute is authorized and before its deadline. A ciphertext that cannot be opened in time is, for the dispute, unavailable. We use AI-rollup disputes as a running example, but nothing in

our model depends on the workload: the same question arises wherever a protocol must open threshold-encrypted inputs on demand.

The failure is silent until it is needed. Custodians may lose shares, fail together when a provider or region goes down, or run broken software, and the key must stay serviceable for a dispute window that lasts days and may outlive the committee that sealed the record. A strategic custodian may also answer every routine check and refuse one targeted dispute; we show that no audit of the kind studied here can detect that, and treat it as a limit rather than a goal. Existing building blocks answer adjacent questions: threshold encryption limits who can decrypt[4], verifiable secret sharing shows that shares were distributed correctly[5,6], proactive sharing refreshes them[7], and proofs of retrievability and data-availability sampling check that data can be retrieved[8,9]. None produces public evidence that enough custodians can serve an authorized dispute now, or states what such evidence implies.

In this article, we study this property, *dispute-key availability* (DKA), and we design and evaluate KEYSTONE, a readiness audit for it that targets non-strategic faults. KEYSTONE seals each record under a fresh key wrapped by a threshold Diffie–Hellman KEM. Routine audits ask a beacon-sampled subset of custodians to apply their shares to a *canary*, a group element hashed from public audit context, and to prove correctness with a Chaum–Pedersen equality-of-discrete-logarithm (DLEQ) proof; each response is structurally a threshold-coin or threshold-signature share on a hashed message[10,11]. An authorized dispute applies the same share operation to the disputed record under a public request-and-deadline transcript. Our contributions are:

1. **A property decomposition** (Section 4): definitions of ciphertext availability, valid-share registration, audit-time readiness, authorized decryptability, deadline liveness, pre-authorization confidentiality and (ε, Δ_d)-DKA, with constructive witnesses for three inferences a deployment must not draw.
2. **A confidentiality-preserving audit** (Section 5): we use threshold-coin-style shares as a sampled readiness test; adapting the standard simulation argument, canary transcripts are simulatable from public shares in the random-oracle model, and a coordinator cannot substitute a production record for a canary. We also show that the plain threshold KEM is malleable across records, so confidentiality against parties who obtain other records' releases requires binding each release to its record.
3. **An exact analysis** (Section 6): full operating characteristics for uniform and stratified samples, showing that small audits cannot separate a committee one share short of t from a live one, the sample sizes and sequential tests that certify a stated safety margin, a placement condition, the detection versus benign-alarm trade-off, and a deadline budget.
4. **An evaluation with labeled evidence** (Section 7) that combines exact analysis, three prespecified simulation cells, microbenchmarks at RFC 7919 group sizes and a 32-custodian emulated testbed, and reports negative results: a high benign alarm rate under the strictest rule, a selective-withholding counterexample and a primary deadline endpoint that no profile met.

None of the primitives is new, and a passed audit does not prove future availability.

Sections 2–7 cover preliminaries, model, properties, protocol, analysis and evaluation; Sections 8–11 cover design consequences, related work, limitations and conclusions.

2 Preliminaries

Groups and sharing. Let $\mathbb{G}$ be a cyclic group of prime order ℓ with generator g in which the decisional Diffie–Hellman (DDH) problem is hard. The prototype uses the order-ℓ subgroup of quadratic residues modulo the RFC 7919 safe primes [12] (ffdhe2048, ffdhe3072), with $\ell = (p-1)/2$ and $g = 4$; q denotes only the acceptance threshold of an audit, and $\binom{a}{b} = 0$ when $b > a$ or $b < 0$. A secret $x \in \mathbb{Z}_\ell$ is Shamir-shared with a polynomial f of degree $t-1$, $f(0) = x$; custodian $i \in \{1, \ldots, n\}$ holds $x_i = f(i)$ and publishes $Y_i = g^{x_i}$ [13], verifiable against Feldman commitments [5]. Any t distinct indices I have Lagrange coefficients λ_i^I with $\sum_{i \in I} \lambda_i^I x_i = x$.

Threshold Diffie–Hellman KEM. With epoch key $PK = g^x$, a sender draws $\rho \leftarrow \mathbb{Z}_\ell$, publishes $C_1 = g^\rho$ and derives a key-encryption key from PK^ρ and the record identifier. Any t partial decryptions $C_1^{x_i}$ determine $C_1^x = \prod_{i \in I} (C_1^{x_i})^{\lambda_i^I}$. This threshold ElGamal KEM [4] is semantically secure under DDH in the random-oracle model but malleable: anyone can turn C_1 into $C_1 g^\gamma$. Shoup and Gennaro's TDH2 binds a label and a proof of knowledge of ρ to the ciphertext, which makes it secure against chosen-ciphertext attacks [14]; Section 5.2 explains when KEYSTONE needs it.

DLEQ proofs, hashing and beacon. A Chaum–Pedersen proof [15] shows $\log_g Y_i = \log_X D_i$: the prover picks $\nu \leftarrow \mathbb{Z}_\ell$, sends $a_1 = g^\nu$, $a_2 = X^\nu$, derives $c = H(g, Y_i, X, D_i, a_1, a_2, \mathsf{ctx})$ by the Fiat–Shamir transform [16] and returns $z = \nu + c\,x_i$; the verifier checks $g^z = a_1 Y_i^c$ and $X^z = a_2 D_i^c$. The protocol is complete, special-sound and honest-verifier zero knowledge, and the request context ctx ties each proof to one request. H_G maps byte strings to $\mathbb{G} \setminus \{1\}$ with domain separation; we model H and H_G as random oracles. A public beacon outputs values that are unpredictable before publication [17].

3 System and threat model

An epoch E has n custodians placed in failure domains (providers, regions, software stacks), a threshold t, a key PK_E and registered public shares $Y_1, \ldots, Y_n$. A rollup worker encrypts each inference record r under a fresh data key K_r with AES-GCM, wraps K_r under the KEM key derived from $(C_1^{(r)}, PK_E)$, and publishes the result to a data-availability (DA) layer. A settlement layer, the *bulletin board*, records registrations, requests, deadlines and evidence and supplies an ordered public clock. A *combiner* inside the authorized destination collects partial decryptions; it learns K_r for records it is authorized to open and is trusted not to disclose it. Dispute partials reach it over authenticated, confidential channels, and the board stores only commitments to responses with their validity status and timestamps, so it never holds enough material to decrypt a record. Canary responses may be public (Lemma 2). Figure 1 shows the three paths.

Assumption 1. Threshold confidentiality. During an epoch the adversary learns fewer than t current shares. DDH is hard in $\mathbb{G}$; H, H_G and the key-derivation function are independent, domain-separated random oracles; and AES-GCM is a secure authenticated-encryption scheme.

Assumption 2. Beacon. An audit's beacon value is unpredictable until its epoch, slot and committee are committed on the bulletin board, and the derived sample is uniform (without

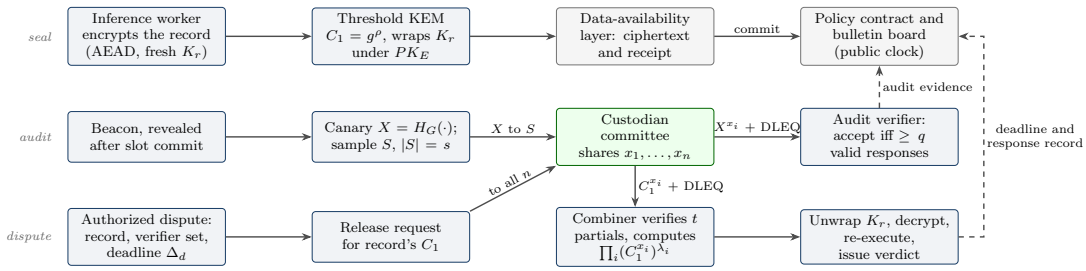


Figure 1. KEYSTONE architecture. Sealing, routine canary auditing and authorized dispute release are separate paths over one custodian committee. The audit uses a beacon-derived canary, never a production ciphertext; only an authorized dispute requests partial decryptions of a record's KEM component C_1. Dashed arrows are public evidence written to the bulletin board.

replacement, or within each domain quota for stratified samples) and independent of other audits' samples. The prototype seeds a pseudorandom generator with a SHA-256 hash of the beacon; we model this as ideal, and a production system should use a cryptographic sampler.

Assumption 3. Bulletin board and data availability. The bulletin board is append-only and totally ordered and exposes each request to custodians within δ_b. The DA layer keeps committed ciphertexts and receipts retrievable.

Assumption 4. Bounded delay, for deadline statements only. Messages between non-faulty parties arrive within δ over channels that authenticate the sending custodian. A non-faulty custodian produces a partial decryption with proof within c_p; the combiner verifies one proof within c_v and combines t partials within c_c.

Detection and confidentiality statements do not use Assumption 4. The adversary may crash custodians individually or by domain, cause share loss, send invalid or equivocal responses, move corruption across epochs, issue unauthorized requests, act as a worker that commits records of its choice and obtains disputes on them, and control *selective* custodians that answer canaries but withhold targeted dispute responses. Corruption is static within an epoch: the adversary fixes the custodians it controls when the epoch starts. Out of scope are production key-generation security, malicious combiners, verifiers or trusted execution environments that leak records they are authorized to open, arbitrary asynchronous partitions, and selective withholding beyond the counterexample of Observation 1. Two limits follow from the model. An audit shows that whoever answered knows x_i, not that the registered operator does, so share pooling by one operator and false failure-domain declarations are invisible to it. And threshold custody replaces the one-honest-challenger assumption of optimistic rollups with an assumption on t of n custodians.

4 Dispute-key availability

Fix an epoch E, a record r with KEM component C_1, and a request time τ on the bulletin board.

Definition 1. Ciphertext availability. $\mathrm{CA}(r, \tau)$ holds if the committed encrypted record and its receipt are retrievable from the DA layer at τ.

Definition 2. Valid-share registration. $\mathrm{VSR}(E)$ holds if the registered public shares are consistent with one degree-$(t-1)$ polynomial whose constant term is the epoch secret, and every admitted custodian initially received the matching share.

Definition 3. Readiness. For a request u, an audit or a dispute on a named record, $\mathsf{Ready}_i(E, \tau, u, \Delta) = 1$ if custodian i can access its current share, perform the operation u requests, produce a valid DLEQ proof and deliver the response by $\tau + \Delta$. Audit readiness and dispute readiness for each record are distinct predicates.

Definition 4. Audit-time readiness. For a sample S of s custodians and threshold $q \le s$, $\mathrm{AKR}(E, S, \tau)$ holds if at least q members of S deliver valid canary responses by the audit deadline.

Definition 5. Authorized decryptability and deadline liveness. After a valid authorization for r at τ, $\mathrm{AD}(r, E, \tau)$ holds if the combiner obtains valid partial decryptions of C_1 from at least t distinct custodians, and $\mathrm{DDL}(r, E, \tau, \Delta_d)$ holds if this happens by $\tau + \Delta_d$.

Definition 6. Pre-authorization confidentiality. PAC holds for r if no probabilistic polynomial-time adversary that satisfies Assumption 1 and observes all audit transcripts, but no dispute releases, distinguishes K_r from a random key with non-negligible advantage. In the stronger PAC^+, the adversary may also commit records of its choice and obtain the release of every record other than r, where a *release* is the set of partial decryptions and the combined value delivered to the authorized destination, which the adversary may control for its own records.

Definition 7. Dispute-key availability. Fix a readiness model, a distribution over custodian failures, share loss and message delays over time. An epoch provides (ε, Δ_d)-DKA under it if for every committed record r and authorization time τ, $\Pr[\mathrm{DDL}(r, E, \tau, \Delta_d)] \ge 1 - \varepsilon$ over the readiness model and the protocol's randomness, and PAC holds for every record before its authorization.

Definition 8. Catastrophic state and detection. An epoch is *catastrophic* at τ if fewer than t custodians are audit-ready. An audit *falsely accepts* if AKR holds in a catastrophic state; its *detection probability* is one minus its false-accept probability. Dispute readiness can be lower than audit readiness (Observation 1).

Remark 1. Timescale and epochs. Deployed optimistic rollups keep dispute windows open for days and bound each interactive move by hours, so DKA chiefly requires the key to stay serviceable throughout the window (ε over the window), while Δ_d matters for per-move timers and short-window designs. A record sealed under PK_E remains disputable after the committee of E rotates, so DKA must hold across epochs: old committees stay serviceable until their records' windows close, or shares are handed over by dynamic proactive sharing [18,19], and audits must cover every epoch with open records.

DKA is conditional and probabilistic. The next observation records what the weaker properties do not imply. Its witnesses are elementary; each blocks a tempting inference, and the design is organized around them.

Observation 1. Non-implications. *Fix* $1 \le t \le n$, $1 \le q \le s \le n$ *and* Δ_d.
1. $\mathrm{CA}(r, \tau)$ *does not imply* $\mathrm{DDL}(r, E, \tau, \Delta_d)$.
2. $\mathrm{AKR}(E, S, \tau)$ *does not imply* $\mathrm{AD}(r, E, \tau')$ *for an authorized dispute at* $\tau' \ge \tau$.
3. For every m, m *accepted audits at times* $\tau_1 < \cdots < \tau_m$ *do not imply* $\mathrm{AD}(r, E, \tau')$ *for* $\tau' > \tau_m$.

Proof. (1) Let the DA layer retain the record while exactly $t-1$ custodians are dispute-ready for r and the others have lost their shares. CA holds, but at most $t-1$ valid partials exist, so AD and DDL fail. (2) Let all custodians be audit-ready and let $w = n - t + 1$ of them answer every canary and withhold every response for r. Every sample has $s \ge q$ valid responses, so AKR holds for every S, while at most $t-1$ custodians answer the dispute. (3) Take two executions identical up to τ_m with all custodians audit-ready, so all m audits accept in both. Afterwards all custodians stay ready in the first and $n - t + 1$ lose their shares in the second. The audit histories are identical but AD holds at τ' only in the first, so without assumptions on how readiness evolves no predicate of the audit history implies AD at τ'; Corollary 1 shows what a static model recovers.

Item (2) is why audit and dispute readiness are separate predicates; item (3) is why any repeated-audit guarantee needs a temporal model.

5 The KEYSTONE protocol

Setup and sealing. The committee is formed under identity, stake and failure-domain placement policy (Observation 2). A deployment would use an authenticated distributed key generation or publicly verifiable secret sharing[6,20,21]; the prototype uses a dealer with Feldman commitments as an explicit substitution boundary. To seal r, the worker draws ρ, computes $C_1 = g^\rho$ and $\kappa = \mathrm{KDF}(PK_E^\rho, \mathsf{id}_r, \mathsf{aad})$, wraps K_r under κ, encrypts the record under K_r with AES-GCM, and publishes an envelope binding epoch, record identifier, ciphertext, metadata and DA pointer.

Readiness audit. The coordinator commits an epoch and audit slot. The next finalized beacon value β fixes a sample S of s custodians, uniform or stratified by domain, and everyone derives the canary $X = H_G(\texttt{KEYSTONE-CANARY} \parallel E \parallel \mathsf{slot} \parallel \beta)$. Each sampled custodian returns $D_i = X^{x_i}$ with a DLEQ proof that $\log_g Y_i = \log_X D_i$, bound to the request context and its index. The verifier checks membership, freshness, uniqueness per index, proof validity and the deadline, and accepts after q valid responses.

Authorized dispute. A dispute starts once the policy contract binds a record, epoch, authorized verifier and deadline Δ_d. Each custodian checks the authorization and returns $C_1^{x_i}$ with a DLEQ proof bound to the dispute. The combiner verifies responses on arrival, discards duplicates and invalid ones, stops at t valid partials, interpolates $C_1^x = PK_E^\rho$, re-derives κ, unwraps K_r and decrypts.

Transcript and evidence. For each request, the board records commitments to valid, invalid, equivocal, stale and unauthorized responses and the absence of missing ones, with timestamps. Invalid proofs and conflicting responses are self-contained cryptographic evidence; attributing a missing response to a custodian additionally requires Assumption 4. The prototype contract is a skeleton: an administrator supplies the sample and canary hash when opening an audit without an on-chain beacon check, records invalid-response evidence after off-chain verification, and can cancel requests. Theorem 1 therefore holds for the prototype only if the administrator samples as the protocol prescribes; deriving the sample on chain from the committed beacon and verifying proofs on chain are needed for a trustless deployment.

5.1 Security of the audit

Lemma 1. Correctness and knowledge soundness. *An honest response passes verification. In the random-oracle model a verifying response satisfies $D_i = X^{x_i}$ except with negligible probability, and the proof is a proof of knowledge: from any party that produces verifying responses for fresh canaries with non-negligible probability, an extractor obtains x_i. Any t verified partials of C_1 with distinct indices I combine to $\prod_{i\in I}(C_1^{x_i})^{\lambda_i^I} = PK_E^{\rho}$.*

Proof. Completeness: $g^z = g^{\nu+cx_i} = a_1 Y_i^c$ and $X^z = a_2 D_i^c$ when $D_i = X^{x_i}$. The protocol is special-sound, since two accepting transcripts with one commitment and different challenges yield x_i; with the Fiat–Shamir hash as a random oracle, the forking lemma turns a party that succeeds with non-negligible probability into an extractor, and value soundness follows[15,16]. Combination: $\prod_{i\in I} C_1^{x_i\lambda_i^I} = C_1^x = g^{\rho x} = PK_E^{\rho}$.

Knowledge soundness makes a passed audit meaningful: whoever passes canary audits with non-negligible probability holds x_i, and under Assumption 1 the adversary cannot obtain an honest custodian's share from public values without solving a discrete logarithm.

Lemma 2. Audit transcripts are simulatable. *Model H_G and H as random oracles. A simulator that knows only g and $Y_1, \ldots, Y_n$ answers all queries to H_G and H and produces the honest custodians' audit responses $(X, (D_i, \pi_i)_{i\in S})$ for any audit context and any S, including $S = \{1, \ldots, n\}$. With at most N simulated proofs and Q oracle queries, simulated and real views are within statistical distance NQ/ℓ, so observing audits changes any adversary's advantage against* PAC*, measured as $|\Pr[1 \mid K_r] - \Pr[1 \mid \mathit{random}]|$, by at most $2NQ/\ell$.*

Proof. The simulator answers every fresh H_G query, from the protocol or the adversary, with g^a for a stored $a \leftarrow \mathbb{Z}_\ell^*$, which is exactly a random oracle onto $\mathbb{G} \setminus \{1\}$. For an honest custodian and canary $X = g^a$ it sets $D_i = Y_i^a = X^{x_i}$, the real response. Each proof is produced by the honest-verifier zero-knowledge simulator: draw $c, z \leftarrow \mathbb{Z}_\ell$, set $a_1 = g^z Y_i^{-c}$ and $a_2 = X^z D_i^{-c}$, and program H at that input to c. Since a_1 is uniform, the input was queried before with probability at most Q/ℓ, and a union bound over N proofs gives the distance. Corrupted custodians' responses come from the adversary. Because H, H_G and the key-derivation function are independent and domain separated, this programming does not interfere with a reduction to the KEM that programs the key-derivation oracle, so an adversary against PAC that sees audits becomes one against the threshold KEM alone, each output probability moving by at most NQ/ℓ.

This is the standard simulation argument for threshold-coin and threshold-signature shares on hashed messages[10,11], which we restate for the audit setting. Lemma 2 holds even for adversarially chosen audit contexts and beacons. It is why the audit can exercise every sampled share without opening any record: combining t canary partials yields X^x for a hash-derived X, which equals a fixed record component only with negligible probability.

Lemma 3. No production substitution. *Model H_G as a random oracle. For a fixed C_1, a coordinator making Q queries to H_G finds an audit context whose canary equals C_1 with probability at most $Q/(\ell - 1)$.*

Proof. Each query returns an independent uniform element of $\mathbb{G} \setminus \{1\}$, equal to the fixed $C_1 \neq 1$ with probability $1/(\ell - 1)$; apply the union bound.

Remark 2. The prototype hashes to an integer modulo p and squares into the quadratic residues; a production system should use a standardized hash-to-curve suite[22]. Binding request context into the Fiat–Shamir hash gives verification-domain binding, not the share isolation of context-dependent threshold decryption[23]. The lemmas do not cover side channels.

5.2 Confidentiality against releases of other records

Lemma 2 concerns audits only. An adversary that can also obtain releases, for example a worker that commits and disputes its own records, defeats the plain KEM.

Proposition 1. Malleability across records. *With the plain threshold Diffie–Hellman KEM,* PAC$^+$ *does not hold: an adversary that obtains the release of one record of its choice recovers K_r for any target record r.*

Proof. For the target's component C_1, the adversary commits a record r' with $C_1' = C_1 g^\gamma$ and obtains its release $C_1'^x = C_1^x PK_E^\gamma$. It computes $PK_E^\rho = C_1^x = C_1'^x PK_E^{-\gamma}$, derives the target's key-encryption key from PK_E^ρ and the target's public record identifier and metadata, and unwraps K_r. Already $\gamma = 0$, re-submitting C_1 under a new identifier, suffices.

The attack does not involve the audit. It is prevented by any threshold KEM whose custodians refuse components not bound to the authorized record, such as TDH2 with the record identifier as label[14] or context-dependent threshold decryption with the record as context[23]. Lemma 2 holds unchanged for such a KEM, since the canary path uses only shares and public values. Our prototype implements the plain KEM and so provides PAC but not PAC$^+$; a deployment that lets untrusted parties obtain releases must bind releases to records, at the cost of one extra proof verification per custodian per dispute.

6 Analysis

Theorem 1. False-accept probability. *Suppose exactly k custodians are audit-ready throughout one audit, the sample S of size s is uniform without replacement and independent of readiness (Assumption 2), and acceptance requires q valid responses. Then, up to the negligible soundness*

error of Lemma 1,

$$\Pr[\mathsf{accept} \mid k] = \sum_{j=q}^{\min(s,k)} \frac{\binom{k}{j}\binom{n-k}{s-j}}{\binom{n}{s}}. \tag{1}$$

This is non-decreasing in k, so over catastrophic states ($k \le t-1$) it is maximized at $k = t-1$, and for $q = s$

$$\varepsilon_a = \max_{k<t} \Pr[\mathsf{accept} \mid k] = \frac{\binom{t-1}{s}}{\binom{n}{s}}. \tag{2}$$

Proof. The number of ready custodians in S is hypergeometric with parameters (n, k, s), and by Lemma 1 the audit accepts exactly when it is at least q, except with negligible probability. For monotonicity, if $R \subset R'$ with $|R| = k$, $|R'| = k+1$, then $|S \cap R| \le |S \cap R'|$ for every S, so acceptance under R implies acceptance under R'. For $q = s$ only $j = s$ contributes, giving $\binom{k}{s}/\binom{n}{s}$, largest at $k = t-1$.

For $n = 32$, $t = 22$, $s = q = 8$, $\varepsilon_a = \binom{21}{8}/\binom{32}{8} = 203{,}490/10{,}518{,}300 = 0.0193$, so a single audit detects the worst catastrophic state with probability 0.9807 under a static, non-selective fault model. Section 6.1 shows why this number alone is misleading. Figure 2 shows detection against s.

Corollary 1. Repeated audits under a static state. *If the ready set is fixed and catastrophic for m audits whose samples are independent (Assumption 2), all m falsely accept with probability at most ε_a^m.*

Proof. Given the fixed ready set, the acceptance events are independent, each with probability at most ε_a by Theorem 1.

By Observation 1(3), the static hypothesis cannot be dropped without a temporal model.

Proposition 2. Stratified samples. *Partition the committee into failure domains $F_1, \ldots, F_h$ of sizes n_j with k_j audit-ready custodians, and sample s_j custodians uniformly without replacement from each F_j, independently, with $\sum_j s_j = s$. Then*

$$\Pr[\mathsf{accept}] = \sum_{\substack{(i_1,\ldots,i_h) \\ i_1+\cdots+i_h \ge q}} \prod_{j=1}^{h} \frac{\binom{k_j}{i_j}\binom{n_j-k_j}{s_j-i_j}}{\binom{n_j}{s_j}}.$$

Proof. The per-domain counts of ready sampled custodians are independent hypergeometric variables with parameters (n_j, k_j, s_j), and the audit accepts if and only if their sum is at least q.

For stratified samples the worst catastrophic state depends on how the $t-1$ ready custodians are spread across domains; Proposition 2 evaluates any placement exactly, and Section 7.2 evaluates stratification by simulation.

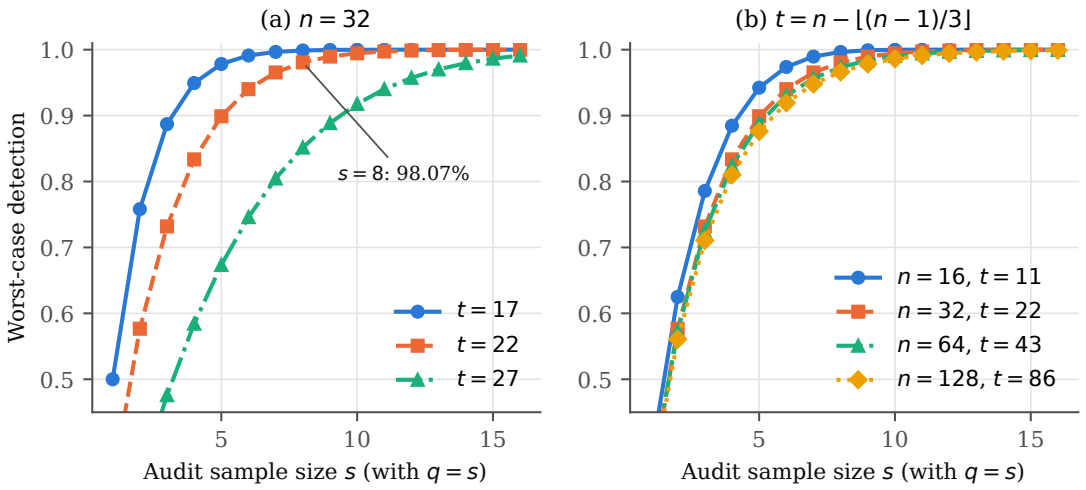


Figure 2. Exact worst-case detection $1 - \binom{t-1}{s} / \binom{n}{s}$ versus sample size with $q = s$. (a) Thresholds $t \in \{17, 22, 27\}$ at $n = 32$; the baseline $s = 8$, $t = 22$ is annotated. (b) Committee sizes 16–128 with $t = n - \lfloor (n-1)/3 \rfloor$. Values for $(n, s) \in \{(16, 4), (32, 8), (64, 12), (128, 16)\}$ are listed in Table 8.

Observation 2. Placement cap. *With no failures other than whole-domain outages, if every domain has at most $n - t$ custodians, no single domain outage prevents reconstruction; if some domain has more than $n - t$, its outage alone does. The cap is tight for single-domain outages.*

Proof. An outage of a domain of size d leaves $n - d$ custodians, and $n - d \geq t$ if and only if $d \leq n - t$.

The baseline places four domains of eight under the cap $n - t = 10$: one outage is survivable, two leave $16 < 22$ custodians.

Observation 3. Benign pass probability. *If each custodian is independently unavailable with probability f and the sample is independent of availability, the audit passes with probability* $\Pr[\mathrm{Bin}(s, 1 - f) \geq q]$.

Proof. Given the sample, its members' availability indicators are independent Bernoulli$(1 - f)$, and the audit passes if and only if at least q are available.

The strict rule $q = s$ that maximizes detection also turns every benign miss into an alarm. For $s = q = 8$ and $f = 0.10$ a healthy committee passes only with probability $0.9^8 = 0.430$, although t of 32 shares are reachable with probability $\Pr[\mathrm{Bin}(32, 0.9) \geq 22] = 0.99983$. Table 1 and Figure 4 give the trade-off. At $s = 8$ no rule achieves both goals: $q = 7$ lowers detection to 85.9% and raises the benign pass rate at $f = 0.10$ to 81.3%. A larger sample helps: $s = 16$, $q = 14$ detects 98.8% with benign pass rates of 95.7% at $f = 0.05$ and 78.9% at $f = 0.10$. We added this analysis after observing the alarm rate in exploratory runs and report it as design guidance, not a prespecified result.

Table 1. Exact acceptance-rule trade-off at $n = 32$, $t = 22$: worst-case detection ($k = t - 1$, Theorem 1) versus the probability that a healthy committee passes under IID unavailability f (Observation 3). Post hoc analysis.

		Worst-case	Benign audit pass under IID unavailability		
s	q	detection	$f = 0.05$	$f = 0.10$	$f = 0.20$
8	6	57.5%	99.4%	96.2%	79.7%
8	7	85.9%	94.3%	81.3%	50.3%
8	8	98.1%	66.3%	43.0%	16.8%
12	10	89.6%	98.0%	88.9%	55.8%
12	11	98.2%	88.2%	65.9%	27.5%
12	12	99.9%	54.0%	28.2%	6.9%
16	13	93.2%	99.3%	93.2%	59.8%
16	14	98.8%	95.7%	78.9%	35.2%
16	15	99.9%	81.1%	51.5%	14.1%

Table 2. Exact operating characteristic: probability that an audit with sample size s and threshold q accepts when k of $n = 32$ custodians are ready ($t = 22$, uniform sampling, Theorem 1).

		Pr[audit accepts] with k ready custodians ($n = 32$, $t = 22$)					
s	q	$k = 21$	$k = 22$	$k = 24$	$k = 26$	$k = 28$	$k = 30$
8	8	1.9%	3.0%	7.0%	14.9%	29.5%	55.6%
8	7	14.1%	19.3%	33.3%	52.4%	74.6%	94.4%
16	14	1.2%	2.7%	11.0%	32.7%	70.0%	100.0%
16	13	6.8%	12.6%	34.3%	67.3%	94.9%	100.0%
24	20	0.1%	0.5%	8.2%	47.6%	100.0%	100.0%

6.1 Operating characteristic and sequential audits

Worst-case detection hides how weak a small audit is near the threshold. Table 2 gives the full operating characteristic of Eq. (1): with $s = q = 8$ the audit accepts a committee one share short of t with probability 1.9%, a live committee with no spare share with 3.0%, and one with four spare shares with only 14.9%. Only the full probe ($s = n$) separates $k = t - 1$ from $k = t$ at 5% error rates, so sampled guarantees must be stated for a safety margin m, testing $k \leq t - 1$ against $k \geq t + m$. With error rates of 5% each, a single audit needs $s = 25$ ($q = 19$) for $m = 4$, $s = 20$ ($q = 16$) for $m = 6$ and $s = 14$ ($q = 12$) for $m = 8$. A sequential probability ratio test across audits[24], valid under the static state of Corollary 1, is far cheaper: for $m = 6$ it decides on average after 2.3 audits for a failed committee and 1.1 for a live one with $s = 16$, $q = 14$, and after 9.8 and 4.6 with $s = q = 8$ (Wald's approximations). Audits should therefore be specified by the margin they certify and the error rates they achieve, not by worst-case detection. This analysis is exact.

Proposition 3. Deadline budget. *Under Assumptions 3 and 4, suppose at least t custodians are non-faulty, hold their shares and follow the protocol, and the combiner is work-conserving, verifies serially and verifies each authenticated index at most once. Then a dispute requested at τ obtains t valid partials by $\tau + \delta_b + c_p + \delta + n\,c_v$, and the key is recovered c_c later. Hence*

$\Delta_d \geq \delta_b + c_p + \delta + n\,c_v$ *suffices for* DDL*, and* $n\,c_v$ *may be replaced by* $t\,c_v$ *if no invalid response precedes the* t*-th valid one.*

Proof. Each of the t correct custodians sees the request by $\tau + \delta_b$ and its response arrives by $T = \tau + \delta_b + c_p + \delta$. Authenticated channels prevent an adversary from occupying a correct custodian's index, so the combiner verifies at most one response per index, at most n in total, and never idles while responses are pending; it therefore finishes by $T + n\,c_v$, or by $T + t\,c_v$ if no invalid response comes first.

On a production settlement layer, δ_b is a block or finality time and can dominate; in our testbed δ_b is negligible and unoptimized verification makes $t\,c_v$ the largest term.

7 Evaluation

We compare the audit with simpler checks (Q1), separate audit outcomes from dispute success under failures (Q2), identify what governs dispute completion (Q3), and measure scaling (Q4).

Method. Table 3 lists every experiment, and each result carries one of four evidence labels. *Exact* values are closed form under a declared model. *Simulation* results come from three cells whose parameters, endpoints and seeds were fixed and hashed before execution; each has 131,072 draws, above the 73,778 that Hoeffding's inequality requires for a half-width of 0.005 at 95% confidence[25]. Where an exact value exists, the cells serve as an implementation check against it. The sampling-policy cell compares uniform and stratified samples on common random numbers. *Microbenchmarks* are single-process medians, and *emulation* runs 32 custodian processes on one host. Proportions carry 95% Wilson intervals[26].

7.1 Readiness baselines (Q1)

Table 4 compares the canary audit with the obvious alternatives. A heartbeat is answered even by a custodian that lost its share, registration is checked once, and decrypting a real record discloses it. A full-committee canary probe detects every static catastrophic state at four times KEYSTONE's per-round cost; it is the case $s = n$ of Theorem 1 and is affordable for one audit per epoch at $n = 32$. Sampling pays off when audits are frequent, committees are large or responses are recorded on chain: at the measured 57,882 gas per response (Table 9), recording responses costs about 0.46 million gas per audit for $s = 8$ and 1.85 million for $s = 32$.

7.2 Robustness under failures (Q2)

In the simulation cells (Table 5), 10% independent unavailability leaves the committee reconstructable in 99.98% of draws while the strict audit passes in only 43.1%: it is a sensitive alarm, not a health score, as Observation 3 predicts. Under 20% domain outages, stratified sampling raised conditional detection of catastrophic draws from 99.89% to 100.00% and lowered the benign pass rate from 37.3% to 34.9%. The prespecified endpoint, the paired difference in detected catastrophes per draw, was 0.00021 with Hoeffding half-width 0.0075, a conservative test that cannot resolve an effect this small; a post hoc exact sign test on the 28 discordant pairs, all favoring stratification, gives $p = 7.5 \times 10^{-9}$, and Proposition 2 gives the exact difference for

Table 3. Experimental configuration. The baseline committee is $n = 32$, $t = 22$, $s = q = 8$ with four failure domains of eight custodians.

Experiment family	Parameters	Execution and evidence label
Static bound (Eq. 2)	$n \in \{16, 32, 64, 128\}$, $t = n - \lfloor (n-1)/3 \rfloor$, $s \in \{1, \ldots, 16\}$, $q = s$; baseline $n = 32, t = 22, s = q = 8$	Exact closed form
Simulation cells	$n = 32, t = 22, s = q = 8$, 4 domains of 8. IID: $p_{\text{off}} = 0.10$. STRAT: $p_{\text{dom}} = 0.20$, $p_{\text{off}} = 0.02$, uniform vs. 2-per-domain quota. SW: $w = 11$	32 seed blocks × 4,096 draws = 131,072 per cell; seeds frozen before execution; simulation
Correlation model	$f \in \{0, 0.025, \ldots, 0.40\}$; IID vs. 2/4/8 equal domains	Exact; declared synthetic model
Acceptance rule	$s \in \{8, 12, 16\}$, $q \in [\lceil s/2 \rceil, s]$, IID $f \in \{0.05, 0.10, 0.20\}$	Exact; post hoc
Cryptographic cost	ffdhe2048, ffdhe3072 (RFC 7919) and the 256-bit prototype group; $n \leq 128$	Local microbenchmark, CPython 3.13, Apple M4 Pro
Deadline testbed	ffdhe2048, $n = 32, t = 22$; 8 profiles × 300 traces; one-way delay lognormal (median 0/40/120 ms); 2% loss; 10 or 11 crashes; one-domain outage; $\Delta_d \in \{0.25, 0.5, 1, 2, 5\}$ s	32 custodian processes on one host, loopback TCP, real cryptography; emulation

any placement. With eleven selective withholders, audits passed in 100.0% of draws and disputes succeeded in 0.0%; this cell instantiates the deterministic witness of Observation 1(2) and only confirms that the implementation reproduces it.

Placement matters more (Figure 3). At per-unit unavailability $f = 0.10$, an IID committee reconstructs with probability 99.98%, four domains of eight (under the cap of Observation 2) with 94.8%, and two domains of sixteen, which violate it, with only 81.0%. These values are exact under a declared synthetic model; field studies show that correlated failures dominate real unavailability [27].

7.3 Deadline liveness in an emulated testbed (Q3)

Thirty-two custodian processes, each an asyncio TCP server holding only its own ffdhe2048 share, and one combiner run on one host. Each trace runs one canary audit and one dispute over a freshly sealed 1 KiB record; the combiner verifies proofs on arrival, stops at $t = 22$ valid partials, combines, unwraps K_r and checks the plaintext. One-way delays are lognormal with the profile's median, loss drops messages without retransmission, and crashed custodians never answer. Each of eight profiles ran 300 traces with seeds fixed in advance.

Table 6 gives the results. No dispute finished in under 3.5 s, so the primary endpoint fixed before the run, success by $\Delta_d = 2$ s, was 0% in every profile; that deadline was set without budgeting for the serial verification term of Proposition 3, which in this prototype alone exceeds

Table 4. Readiness checks compared. Costs are measured medians for ffdhe2048 in the prototype (Table 7); bytes are canonical request plus response transcripts.

Readiness check	Exercises the secret share?	Touches production ciphertext?	Detects a static state with $k = t - 1$ ready shares	Cost per check ($n = 32$, ffdhe2048, measured)
Liveness heartbeat	No	No	No: a node that lost its share still answers	1 message per node; no group operations
Share registration (Feldman/PVSS check)	Once, at setup	No	No: says nothing about later share loss	One-time $O(nt)$ exponentiations at epoch setup
Full-committee canary probe	Yes	No (canary)	Yes, with probability 1	32 proofs + 32 verifications; 8501 ms CPU; 36,991 B
KEYSTONE sampled canary ($s = q = 8$)	Yes, for the sampled members	No (canary)	Yes, with probability 0.9807 (Eq. 2)	8 proofs + 8 verifications; 2125 ms CPU; 9,415 B
Decrypt a real record as a test	Yes	Yes: exposes a production record	Yes	t verifications plus a real disclosure; excluded by design

Table 5. Simulation cells, 131,072 draws each, 95% Wilson intervals. Reconstructable means at least t custodians can serve the dispute.

Scenario	Audit pass	Reconstructable
IID 10% unavailable	43.1% [42.8, 43.3]	99.98% [99.97, 99.99]
Domain outage 20%, uniform sample	37.3% [37.0, 37.6]	81.3% [81.1, 81.6]
Domain outage 20%, stratified sample	34.9% [34.7, 35.2]	81.3% [81.1, 81.6]
11 selective withholders	100.0% [99.997, 100.000]	0.0% [0.000, 0.003]

2 s. Within 5 s every reconstructable dispute succeeded except one control trace (299 of 300), with medians from 3.69 to 3.88 s and a slowest success of 4.48 s; that trace timed out with 19 valid partials during unrelated load on the shared host. Without injected delay the median was 3818 ms, of which 22 serial verifications at 135.4 ms (Table 7) account for about 2,980 ms; the remainder was not instrumented. A 120 ms-median delay moved the median only to 3875 ms, because responses arrive while earlier ones are still being verified, and a heavy-tailed law ($\sigma = 1.2$) did not lengthen completion (95th percentile 3850 ms). Ten crashes and a domain outage finished slightly faster, plausibly because fewer processes competed for the CPU; eleven crashes defeated every dispute (0/300 succeeded), since $21 < t$ custodians remain. In this unoptimized single-host setting, completion was bounded by computation rather than emulated network delay; we did not measure optimized or batched verification.

Audit pass rates on real cryptography matched the exact expectations (Eq. (1) for crashes and outages, Observation 3 for loss): 2.3% against 3.0% with ten crashes, 5.7% against 7.0% with one domain down, and 71.7% against 72.4% with 2% loss, while the dispute succeeded in every trace of these profiles. A single audit took 1.24–1.43 s at the median. Under the heavy-tailed law

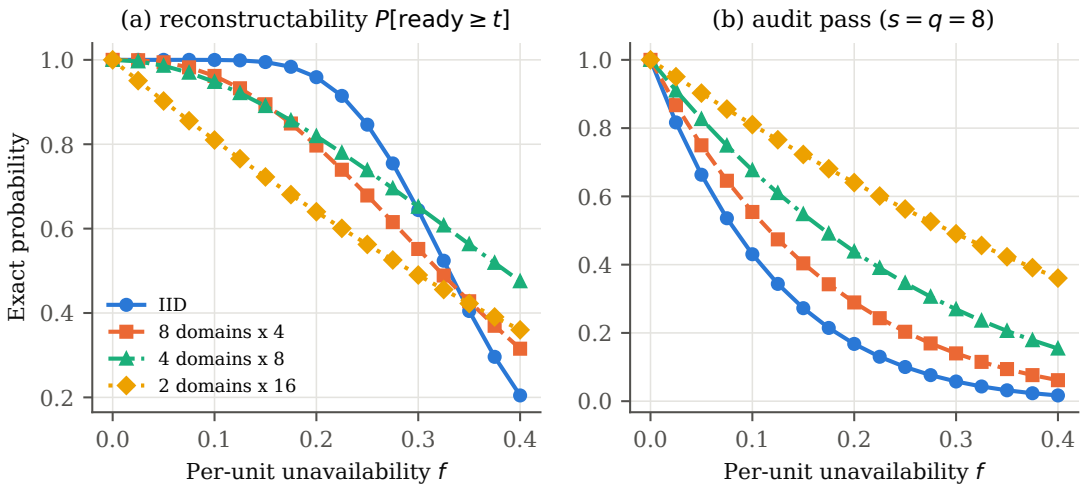


Figure 3. IID versus domain-correlated unavailability at $n = 32$, $t = 22$. A domain fails with probability f and takes all its members; in the IID model each custodian fails independently with probability f. (a) Reconstructability. (b) Audit pass with $s = q = 8$. Exact; a 20,000-draw Monte Carlo check agrees within 0.009.

Table 6. Emulated testbed, 32 custodian processes on one host with real ffdhe2048 cryptography, 300 traces per profile, 95% Wilson intervals. Audit pass is $q = 8$ valid responses within the 6 s timeout; dispute columns require the correct plaintext by the stated deadline; times are over successful disputes.

Profile	Audit pass	By $\Delta_d = 2$ s	By 5 s	Median	p95
control	100.0% [98.7, 100.0]	0.0% [0.0, 1.3]	99.7%	3818 ms	3936 ms
regional	100.0% [98.7, 100.0]	0.0% [0.0, 1.3]	100.0%	3841 ms	3908 ms
global	100.0% [98.7, 100.0]	0.0% [0.0, 1.3]	100.0%	3875 ms	4016 ms
global loss2	71.7% [66.3, 76.5]	0.0% [0.0, 1.3]	100.0%	3829 ms	3886 ms
global crash10	2.3% [1.1, 4.7]	0.0% [0.0, 1.3]	100.0%	3692 ms	3775 ms
global crash11	1.7% [0.7, 3.8]	0.0% [0.0, 1.3]	0.0%	–	–
global domain outage	5.7% [3.6, 8.9]	0.0% [0.0, 1.3]	100.0%	3700 ms	3780 ms
heavy tail	98.3% [96.2, 99.3]	0.0% [0.0, 1.3]	100.0%	3772 ms	3850 ms

the audit failed in 5 of 300 traces, each with 7 of 8 responses verified before the 6 s timeout, consistent with the lognormal tail.

7.4 Performance and scalability (Q4)

Table 7 gives per-operation costs of the pure-Python prototype; optimized libraries, in particular elliptic-curve groups, are substantially faster. At ffdhe2048 a partial decryption with proof takes 130.3 ms and its verification 135.4 ms, and verifying 22 partials and combining takes 3213 ms

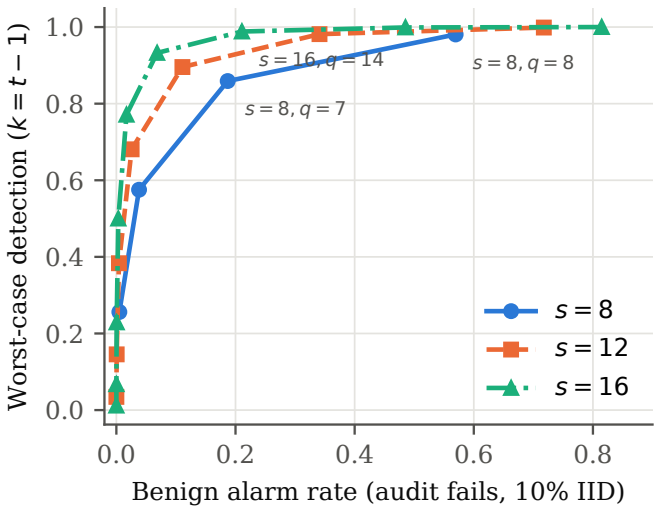


Figure 4. Detection versus benign alarm rate (the audit fails while the committee is healthy, $f = 0.10$) for each q, one curve per sample size. Exact values from Table 1 and its full source grid.

Table 7. Median wall time per operation, CPython 3.13 on an Apple M4 Pro; RFC 7919 groups[12] with $\ell = (p - 1)/2$ and generator 4. Pure-Python reference implementation.

Operation (median, ms)	256-bit prototype	ffdhe2048	ffdhe3072
Canary derivation H_G	0.08	16.99	54.32
Partial decryption + DLEQ proof	0.58	130.30	420.54
DLEQ verification	0.75	135.36	430.58
Seal 1 KiB record	0.22	48.32	156.48
Partial response size (bytes)	253	1149	1661

(10201 ms at ffdhe3072). The 256-bit group is listed for reference only and is not a secure size. Audit cost grows with s and dispute cost with t (Table 8); Table 9 gives gas costs.

8 Discussion

Availability is layered: data availability concerns encrypted bytes, share registration the key's formation, a readiness audit present share access, and a dispute whether t custodians serve a specific target in time; the evaluation separates each layer.

Four design consequences follow. First, specify audits by the safety margin they certify: choose s and q, or a sequential test across audits, for target error rates at $k \leq t - 1$ and $k \geq t + m$ (Section 6.1), which favors larger samples with $q < s$. Second, placement under the cap $n - t$ matters more than the sampling policy. Third, deadlines must budget for the ledger delay and for verifying up to n proofs, or t when no invalid response comes first (Proposition 3); optimized

Table 8. Scalability with $t = n - \lfloor (n-1)/3 \rfloor$. Detection is exact (Eq. (2), $q = s$). Audit CPU is s proofs plus s verifications; bytes are one request plus s responses. Dispute time is measured verification of t partials plus combination.

				Audit, ffdhe2048		Dispute: verify t + combine	
n	t	s	Detection	CPU	Bytes	ffdhe2048	ffdhe3072
16	11	4	88.46%	1063 ms	4,819	1585 ms	5040 ms
32	22	8	98.07%	2125 ms	9,415	3213 ms	10201 ms
64	43	12	99.66%	3188 ms	14,011	6230 ms	20086 ms
128	86	16	99.92%	4250 ms	18,607	12534 ms	40201 ms

Table 9. Bulletin-board gas per operation, measured with Foundry 1.5.1 and Solidity 0.8.24 (test-body gas; fixture setup excluded), in the small harness committee named in the scope column; values are deterministic.

Bulletin-board operation	Gas	Harness scope
Register epoch	223,629	n=5, t=3; duplicate-identity validation enabled
Open audit	125,107	three sampled members; q=3
Submit audit response	57,882	first commitment from one sampled custodian
Mark invalid response	27,704	admin records a non-zero evidence hash
Record equivocation	51,710	second conflicting commitment from one custodian
Finalize audit	29,142	two responses for a three-member sample after deadline
Open dispute	144,149	n=5, t=3; all members eligible
Cancel request	5,709	admin cancels one open audit

or batched verification is the first improvement. Fourth, if untrusted parties can obtain releases, the dispute path must bind each release to its record (Proposition 1).

Invalid proofs and equivocation are transferable evidence, whereas non-response proves only absence and its attribution depends on timing[28], so slashing should be reserved for transferable evidence. Finally, the audit cannot detect selective withholding, because canaries are unrelated to records by design (Lemma 2). Making canaries indistinguishable from record components would stop observers from checking that an audit opened no production record (Lemma 3), so resistance needs a different mechanism, such as unpredictable dispute routing or economic commitments enforced by the deadline transcript.

9 Related work

Table 10 summarizes how KEYSTONE relates to other checks of availability or readiness.

Optimistic and verifiable computation. Optimistic rollups rely on dispute protocols in which an honest challenger forces correct results on chain[1]; opML applies this to machine-learning inference[2], and EigenAI re-executes private inference in a trusted environment after a threshold key release[3]. These systems assume the challenger can read the disputed inputs. With encrypted inputs that assumption becomes the requirement that keys be serviceable before the deadline, which KEYSTONE isolates and audits without changing the dispute game.

Threshold decryption and key management. Threshold cryptosystems[4], their chosen-ciphertext-secure variants with verifiable shares[14] and DLEQ proofs[15,16] provide the release mechanism.

Table 10. Mechanisms that check some form of availability or readiness. "Secret-key operation" means the check makes a custodian apply its secret share. Entries describe the mechanism class, not every system in it.

Mechanism	What is checked	Secret-key operation	Deadline evidence
Liveness heartbeat	A process answers	No	No
Threshold-wallet heartbeat (tBTC v2)[29]	The full signing group signs a heartbeat message	Yes (signing)	A failed heartbeat triggers fund migration
Proof of custody[30]	A validator holds the data it attests to	Secret-dependent bit	Challenge and penalty
PoR and PDP[8,31,32]	A server still stores a file	No	No
Data-availability sampling[9,33,34]	Published data are retrievable	No	No
VSS, PVSS and DKG transcripts[5,6,20]	Shares were distributed consistently	At setup only	No
Threshold-release systems[35–37]	A release is performed when requested	At release only	System-specific
KEYSTONE	q of s sampled custodians can apply their shares now; t custodians serve a dispute by Δ_d	Before release (canary) and at release	Bulletin board

Verifiable and publicly verifiable secret sharing and distributed key generation[5,6,20,21] make shares consistent when created, and proactive and dynamic sharing refreshes them[7,19]. A refresh, including dynamic-committee resharing[18], also exercises every share, but it is infrequent and costly, and shares can be lost between refreshes. A canary response is structurally a threshold-coin share[10] or a threshold-signature share on a hashed message[11]; our contribution is not this primitive but its use as a sampled readiness test, with the properties it can and cannot certify. Context-dependent threshold decryption isolates shares across contexts[23]. KEYSTONE uses these primitives unchanged and asks whether enough custodians can apply their shares *now*.

Threshold release on ledgers. CALYPSO releases on-ledger secrets to authorized readers through a threshold committee with an auditable access log[35], and ETHTID provides threshold disclosure on Ethereum[36]. Encrypted mempools face the same operational concerns of committees, epochs, collusion and timely decryption[37–39]. FairBlock applies identity-based encryption so that a committee exchanges a fixed number of decryption shares per block, independent of the number of transactions[40], and F3B, building on CALYPSO, has a secret-management committee release a per-transaction key once the transaction is finalized[41]. In both, timely decryption depends on a threshold of committee members serving the release, which is the property DKA makes explicit; the published descriptions we found specify the release path but no pre-release readiness audit. UQ-Vote combines a consensus-finalized bulletin board with consensus-authorized threshold decryption and evaluates liveness under faults[42]. Authorization by finalized state, public request logs and decryption deadlines are thus established, and we do not claim them. These systems specify the release path; KEYSTONE adds the deadline-bound property that release must satisfy for disputes, a pre-release audit that measures it without opening records, and an analysis of what passing it implies.

Proofs of storage, custody and heartbeats. Proofs of retrievability and provable data possession spot-check random parts of a stored file and bound the probability that a server missing data passes[8,31]; HAIL extends this to servers under a mobile adversary[32]. KEYSTONE's audit belongs to this family: the DLEQ proof is a proof of knowledge (Lemma 1), and Theorem 1 plays the role of their sampling bounds. It differs in that possession is not service, a gap KEYSTONE makes explicit (Observation 1); in that confidentiality forces canary responses to be useless for decrypting records (Lemmas 2 and 3); and in that DKA is a multi-party, threshold, target-specific and deadline-bound property. Proof-of-custody proposals make validators commit to a secret-dependent bit of attested data so lazy validators can be penalized[30]. The closest deployed analogue is the tBTC v2 heartbeat, in which each threshold wallet's signing group signs periodic messages and funds leave a wallet that fails one[29]; KEYSTONE instead samples custodians with exact operating characteristics, audits decryption shares under a confidentiality constraint, and ties readiness to dispute deadlines. Data-availability proofs and sampling[9,33,34] concern public data, which KEYSTONE takes as given.

Accountability. Slashing conditions[43] and BFT forensics[44] attribute safety violations with transferable evidence, threshold traitor tracing addresses coalitions that leak decryption capability[45], and attributing liveness failures depends on timing[28].

10 Threats to validity and limitations

Novelty. The components are not new; our claim covers the property decomposition, the confidentiality-preserving audit and its analysis, and the labeled evaluation, and it rests on a bounded literature search.

Internal validity. The simulation design was hashed before execution, but the hash is self-recorded and the design was not reviewed by an independent party; the acceptance-rule and operating-characteristic analyses were added afterwards. The testbed shares one CPU among 33 processes, which explains the slightly faster crash profiles and the one control timeout; both are reported, not discarded.

Construct and external validity. The audit measures present sampled serviceability, not future cooperation, and repeated-audit bounds need a static or modeled readiness process. Failure models are synthetic, the testbed uses loopback TCP with synthetic delays, geo-distributed effects and clock skew are not measured, and timings reflect pure Python.

Prototype scope. A dealer replaces distributed key generation; the plain KEM provides PAC but not PAC^{+}; the contract trusts an administrator for sampling, invalid-response evidence and cancellation; neither code nor contract has been externally audited; and the lemmas hold in the random-oracle model with static corruption.

11 Conclusion and future directions

Ciphertext availability does not ensure that the threshold service needed for an encrypted-rollup dispute answers in time. We defined dispute-key availability and the weaker properties it is confused with, showed that a beacon-derived canary audit exercises real shares while its

transcripts are simulatable from public data, gave its exact operating characteristics, the sample sizes and sequential tests that certify a safety margin, a placement condition and a deadline budget, and showed that confidentiality against parties who obtain other releases requires binding releases to records. The evaluation confirmed each separation: a small strict audit barely distinguishes a failed committee from a live one and alarms often on healthy committees, selective withholders pass every audit and defeat every dispute, and in the emulated committee all but one reconstructable dispute completed within five seconds, bounded by serial proof verification in our unoptimized prototype. Future work should address selective withholding, which the audit provably cannot detect alone; temporal readiness models calibrated on operational data; and a production instantiation with distributed key generation, record-bound releases, optimized verification and realistic ledger delays, evaluated across regions.

Statements and Declarations

Ethical considerations. This article reports non-human computational research using synthetic data, simulation, locally generated prototype outputs and smart-contract tests. It involves no human participants, personal data, production secrets or live systems, so ethical approval was not required.

Consent to participate. Not applicable.

Consent for publication. Not applicable.

Declaration of conflicting interest. The author(s) declared no potential conflicts of interest with respect to the research, authorship and/or publication of this article.

Funding statement. The author(s) received no financial support for the research, authorship and/or publication of this article.

Data availability. Source code, frozen designs with SHA-256 hashes, seed schedules, raw per-block and per-trace outputs, and the scripts that generate every table, figure and in-text number are provided as anonymized supplementary material for review. A public repository link is given on the title page and will be included in the published article.

Declaration of AI use. AI assistance was used for drafting text, integrating code, scripting experiments and generating figures. The author(s) verified the scientific content, the proofs and every reported number against the generated outputs, and take full responsibility for the article.

References

1. Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S. Matthew Weinberg, and Edward W. Felten. Arbitrum: Scalable, private smart contracts. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1353–1370. USENIX Association, 2018. ISBN 978-1-939133-04-5. URL `https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner`.
2. KD Conway, Cathie So, Xiaohang Yu, and Kartin Wong. opML: Optimistic machine learning on blockchain. arXiv preprint arXiv:2401.17555, 2024. URL `https://arxiv.org/abs/2401.17555`.

3. David Ribeiro Alves, Vishnu Patankar, Matheus Pereira, Jamie Stephens, Nima Vaziri, and Sreeram Kannan. EigenAI: Deterministic Inference, Verifiable Results. *arXiv preprint arXiv:2602.00182*, 2026. URL `https://arxiv.org/abs/2602.00182`.
4. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In *Advances in Cryptology – CRYPTO '89 Proceedings*, Lecture Notes in Computer Science, pages 307–315. Springer, 1990. doi: 10.1007/0-387-34805-0_28.
5. Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 427–438, 1987. doi: 10.1109/SFCS.1987.4.
6. Berry Schoenmakers. A simple publicly verifiable secret sharing scheme and its application to electronic voting. In *CRYPTO*, pages 148–164, 1999. doi: 10.1007/3-540-48405-1_10.
7. Amir Herzberg, Stanislaw Jarecki, Hugo Krawczyk, and Moti Yung. Proactive secret sharing or: How to cope with perpetual leakage. In *CRYPTO*, pages 339–352, 1995.
8. Ari Juels and Burton S. Kaliski, Jr. PORs: Proofs of retrievability for large files. In *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS '07)*, pages 584–597. ACM, 2007. doi: 10.1145/1315245.1315317.
9. Mustafa Al-Bassam, Alberto Sonnino, Vitalik Buterin, and Ismail Khoffi. Fraud and data availability proofs: Detecting invalid blocks in light clients. In *Financial Cryptography and Data Security (FC 2021)*, Lecture Notes in Computer Science, pages 279–298. Springer, 2021. doi: 10.1007/978-3-662-64331-0_15.
10. Christian Cachin, Klaus Kursawe, and Victor Shoup. Random oracles in Constantinople: Practical asynchronous Byzantine agreement using cryptography. *Journal of Cryptology*, 18(3):219–246, 2005. doi: 10.1007/s00145-005-0318-0.
11. Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In *Public Key Cryptography – PKC 2003*, Lecture Notes in Computer Science, pages 31–46. Springer, 2003. doi: 10.1007/3-540-36288-6_3.
12. Daniel Kahn Gillmor. Negotiated finite field Diffie-Hellman ephemeral parameters for transport layer security (TLS). Request for Comments 7919, Internet Engineering Task Force, 2016. URL `https://www.rfc-editor.org/rfc/rfc7919`.
13. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
14. Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. *Journal of Cryptology*, 15(2):75–96, 2002. doi: 10.1007/s00145-001-0020-9.
15. David Chaum and Torben Pryds Pedersen. Wallet databases with observers. In *Advances in Cryptology – CRYPTO '92*, pages 89–105. Springer, 1993. doi: 10.1007/3-540-48071-4_7.
16. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – CRYPTO '86*, Lecture Notes in Computer Science, pages 186–194. Springer, 1987. doi: 10.1007/3-540-47721-7_12.
17. Sourav Das, Vinith Krishnan, Irene Miriam Isaac, and Ling Ren. SPURT: Scalable distributed randomness beacon with transparent setup. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2502–2517. IEEE, 2022. doi: 10.1109/SP46214.2022.9833580.
18. Sai Krishna Deepak Maram, Fan Zhang, Lun Wang, Andrew Low, Yupeng Zhang, Ari Juels, and Dawn Song. CHURP: Dynamic-committee proactive secret sharing. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*, pages 2369–2386. ACM,

2019. doi: 10.1145/3319535.3363203.

19. Thomas Yurek, Zhuolun Xiang, Yu Xia, and Andrew Miller. Long live the honey badger: Robust asynchronous DPSS and its applications. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5413–5430. USENIX Association, 2023. URL `https://www.usenix.org/conference/usenixsecurity23/presentation/yurek`.
20. Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20(1):51–83, 2007. doi: 10.1007/s00145-006-0347-3.
21. Sourav Das, Thomas Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous distributed key generation. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2518–2534. IEEE, 2022. doi: 10.1109/SP46214.2022.9833584.
22. Armando Faz-Hernandez, Sam Scott, Nick Sullivan, Riad S. Wahby, and Christopher A. Wood. Hashing to elliptic curves. Technical Report RFC 9380, Internet Engineering Task Force, 2023. URL `https://www.rfc-editor.org/rfc/rfc9380`.
23. Dan Boneh, Benedikt Bünz, Kartik Nayak, Lior Rotem, and Victor Shoup. Context-dependent threshold decryption and its applications. Cryptology ePrint Archive, Paper 2025/279; extended abstract in ASIACRYPT 2025, 2025. URL `https://eprint.iacr.org/2025/279`.
24. Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945. doi: 10.1214/aoms/1177731118.
25. Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830.
26. Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
27. Daniel Ford, François Labelle, Florentina I. Popovici, Murray Stokely, Van-Anh Truong, Luiz Barroso, Carrie Grimes, and Sean Quinlan. Availability in globally distributed storage systems. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. USENIX Association, 2010. URL `https://www.usenix.org/conference/osdi10/availability-globally-distributed-storage-systems`.
28. Andrew Lewis-Pye, Joachim Neu, Tim Roughgarden, and Luca Zanolini. Accountable liveness. *arXiv preprint arXiv:2504.12218*, 2025. URL `https://arxiv.org/abs/2504.12218`.
29. Threshold Network. tBTC v2 bridge: Heartbeat and moving funds. Developer documentation, 2026. URL `https://docs.threshold.network/app-development/tbtc-v2/tbtc-contracts-api/tbtc-v2-api/heartbeat`. Accessed 14 October 2026.
30. Dankrad Feist. Proofs of custody. Blog post, 2021. URL `https://dankradfeist.de/ethereum/2021/09/30/proofs-of-custody.html`. Accessed 14 October 2026.
31. Giuseppe Ateniese, Randal Burns, Reza Curtmola, Joseph Herring, Lea Kissner, Zachary Peterson, and Dawn Song. Provable data possession at untrusted stores. In *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS '07)*, pages 598–609. ACM, 2007. doi: 10.1145/1315245.1315318.
32. Kevin D. Bowers, Ari Juels, and Alina Oprea. HAIL: A high-availability and integrity layer for cloud storage. In *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS '09)*, pages 187–198. ACM, 2009. doi: 10.1145/1653662.1653686.

33. Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. Foundations of data availability sampling. *IACR Communications in Cryptology*, 1(4), 2025. doi: 10.62056/a09qudhdj.
34. Danny Ryan, Dankrad Feist, Francesco D'Amato, Hsiao-Wei Wang, and Alex Stokes. EIP-7594: PeerDAS – Peer Data Availability Sampling. Ethereum Improvement Proposal, 2024. URL `https://eips.ethereum.org/EIPS/eip-7594`.
35. Eleftherios Kokoris-Kogias, Enis Ceyhun Alp, Linus Gasser, Philipp Jovanovic, Ewa Syta, and Bryan Ford. CALYPSO: Private data management for decentralized ledgers. *Proceedings of the VLDB Endowment*, 14(4):586–599, 2020. doi: 10.14778/3436905.3436917.
36. Oliver Stengele, Markus Raiber, Jörn Müller-Quade, and Hannes Hartenstein. ETHTID: Deployable threshold information disclosure on Ethereum. In *2021 3rd International Conference on Blockchain Computing and Applications (BCCA)*, pages 127–134, 2021. doi: 10.1109/BCCA53669.2021.9657019. URL `https://arxiv.org/abs/2107.01600`.
37. Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Paper 2022/898, 2022. URL `https://eprint.iacr.org/2022/898`.
38. Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025. URL `https://www.usenix.org/conference/usenixsecurity25/presentation/bormet`.
39. Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3513–3529. USENIX Association, 2024. ISBN 978-1-939133-44-1. URL `https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri`.
40. Peyman Momeni, Sergey Gorbunov, and Bohan Zhang. FairBlock: Preventing blockchain front-running with minimal overheads. In *Security and Privacy in Communication Networks (SecureComm 2022)*, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, pages 250–271. Springer, 2023. doi: 10.1007/978-3-031-25538-0_14.
41. Haoqian Zhang, Louis-Henri Merino, Ziyan Qu, Mahsa Bastankhah, Vero Estrada-Galiñanes, and Bryan Ford. F3B: A low-overhead blockchain architecture with per-transaction front-running protection. In *5th Conference on Advances in Financial Technologies (AFT 2023)*, volume 282 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 3:1–3:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi: 10.4230/LIPIcs.AFT.2023.3.
42. Jingyu Liu, Jingjing Rao, and Tetsutaro Uehara. UQ-Vote: Consensus-authorized threshold decryption for end-to-end verifiable elections. *IEEE Access*, 14:128905–128916, 2026. doi: 10.1109/ACCESS.2026.3725080.
43. Vitalik Buterin and Virgil Griffith. Casper the friendly finality gadget. arXiv preprint arXiv:1710.09437, 2017. URL `https://arxiv.org/abs/1710.09437`.
44. Peiyao Sheng, Gerui Wang, Kartik Nayak, Sreeram Kannan, and Pramod Viswanath. BFT protocol forensics. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*, pages 1722–1743. ACM, 2021. doi: 10.1145/3460120.3484566.
45. Dan Boneh, Aditi Partap, and Lior Rotem. Accountability for misbehavior in threshold decryption via threshold traitor tracing. In *Advances in Cryptology – CRYPTO 2024*, Lecture Notes in Computer Science, pages 317–351. Springer, 2024. doi: 10.1007/978-3-031-68394-7_11.